VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

ADVANCED SOFTWARE ENGINEERING (CO3065)

ASSIGNMENT REPORT

CHILDREN'S LOCATION MONITORING SYSTEM (CLMS)

CC01 — GROUP 1

SEMESTER 252

Instructor: TRƯƠNG TUẤN ANH

Division of Work

Name	Student ID	Task	Contribution
Nguyễn Trịnh Tiến Đạt	2252147	Backend services: REST API endpoints (children, safe zones, location, dashboard), MySQL schema design, GPS validation and ingest pipeline	100%
Arsyad Ghudzamir AFIF	2560139	Report documentation — system description and dependability analysis (Q1–Q7), hazard analysis	100%
Cao Quế Phương	2252652	Report documentation — system description; use-case and architecture diagrams	100%
Đặng Minh Khang	2252287	Arduino tracker hardware setup, Arduino IoT Cloud integration (multi-tenant client credentials), field testing of GPS pipeline, Frontend UI/UX	100%
Đặng Ngọc Phú	2252617	Authentication module (Google OAuth + email OTP + JWT), geofencing engine and zone-transition logging, SOS feature; security analysis (Q13–Q16); Report documentation; demo video and presentation	100%

Question 1: Finish the description for the system

Answer 1:

A company is going to build a smart system (CLMS) to provide services to parents for monitoring their children. CLMS uses a child device — a mobile application or an embedded GPS tracker connected through a cloud IoT broker — to collect GPS location data and forward it periodically to a backend server. Parents access this information through a web interface to view the child's current location on a map. The system uses periodic location tracking rather than continuous high-frequency tracking to reduce complexity, network usage, and battery drain on the child device.

CLMS stores recent historical location data so parents can review recent movements. The system is designed with Google-based sign-in followed by a one-time password (OTP) sent to the parent's email as a second factor, ensuring that only authorised parents can access a child's information.

The system provides a multi-zone geofencing feature, allowing parents to define safe areas (such as home or school) as either circles or rectangles, and to bind each zone to one or more children. When a child crosses the boundary of one of these zones, the system records an enter/exit event and surfaces it to the parent through the dashboard's alert feed. An emergency (SOS) function is also included, enabling the child to manually send an alert to their parent from the child-side application; the parent receives the alert in their notification feed alongside zone events.

Question 2: Draw the usecase diagram for CLMS system; design your system architecture?

1. Use case diagram

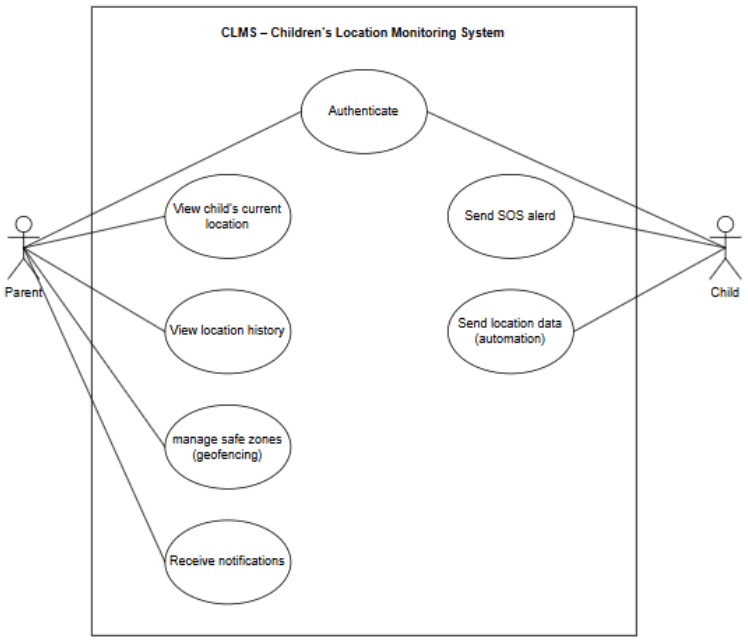


Figure: Overview use case diagram

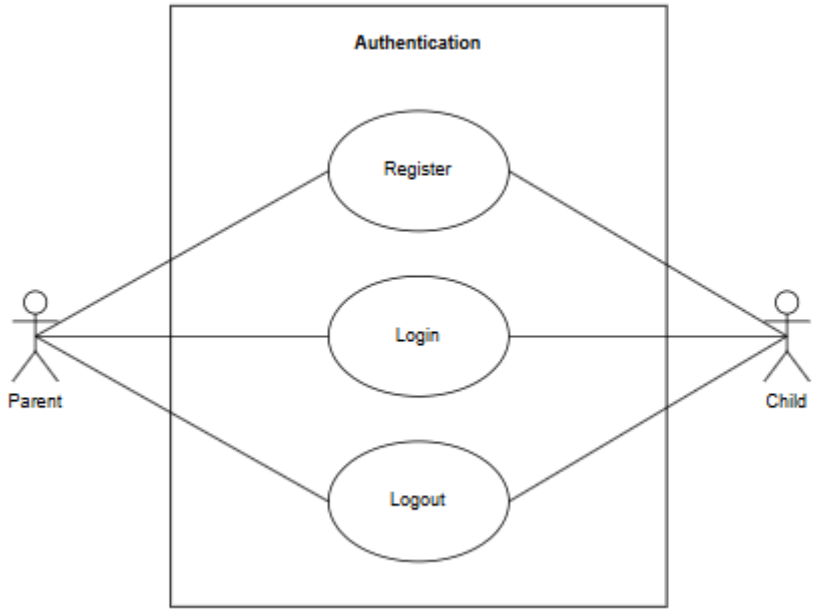


Figure: Authentication use case diagram

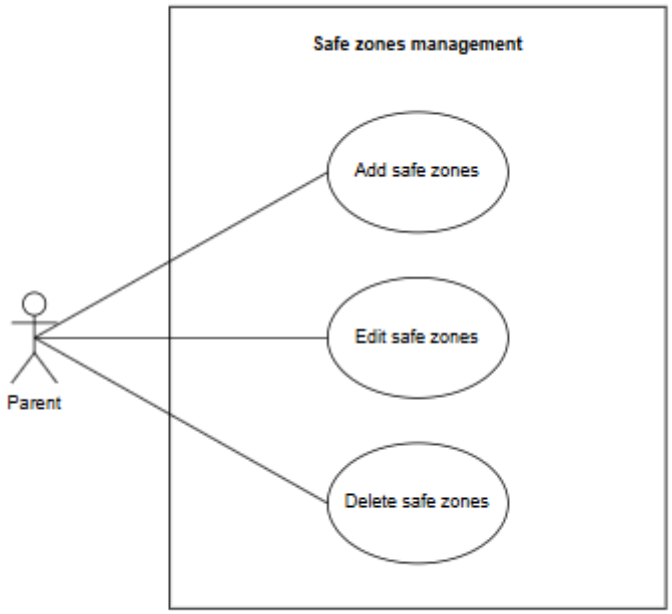


Figure: Safe zones management use case diagram

The figure shows the layered architecture of CLMS. The Presentation Layer handles user interaction through a parent-facing web application and a lightweight child-facing page; raw GPS data is collected by a mobile app or an embedded child device. The Service Layer authenticates parents (Google OAuth + email-OTP MFA), exposes the REST API, and runs a periodic poller that fetches location updates from the cloud IoT

broker. The Business (Domain) Layer contains the core rules: per-child geofence evaluation (circle and rectangle shapes), zone-transition diffing that produces enter/exit events, and location-ownership logic. The Data Layer persists children, safe zones, latest locations, location history, zone-event logs, and user accounts in MySQL, and integrates with external services — the cloud IoT broker for device ingest, Gmail SMTP for OTP delivery, and an external map tile provider rendered client-side.

2. Architecture diagram

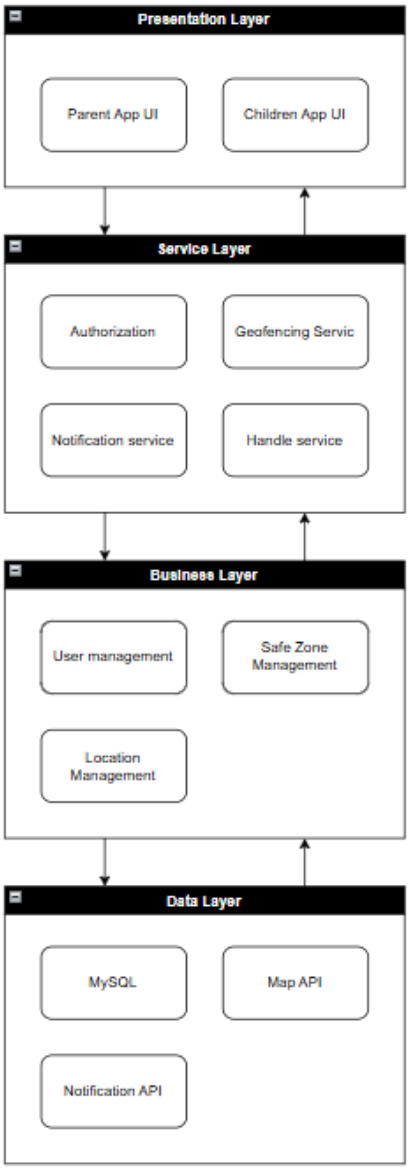


Figure: Overview architecture diagram

The figure shows the layered architecture of CLMS. The Presentation Layer handles user interaction through a parent-facing web application and a lightweight child-facing page; raw GPS data is collected by a mobile app or an embedded child device. The Service Layer authenticates parents (Google OAuth + email-OTP MFA), exposes the

REST API, and runs a periodic poller that fetches location updates from the cloud IoT broker. The Business (Domain) Layer contains the core rules: per-child geofence evaluation (circle and rectangle shapes), zone-transition diffing that produces enter/exit events, and location-ownership logic. The Data Layer persists children, safe zones, latest locations, location history, zone-event logs, and user accounts in MySQL, and integrates with external services — the cloud IoT broker for device ingest, Gmail SMTP for OTP delivery, and an external map tile provider rendered client-side.

Question 3: In order to make the system dependable, what should you do?

Answer 3:

To make the CLMS system dependable, the following measures should be taken:

1. Reliability and Fault Tolerance

Design the system so it continues to work even when some components fail. This includes using backup servers, database replication, and auomatic recovery mechanism to prevent data loss or service downtime.

2. Security and Privacy Protection

Implement strong authentication, secure communication, and proper access control to ensure that only authorized parents can access children's location data.

3. Data Accuracy and Validation

Validate GPS data before storing or displaying it to reduce errors caused by weak signals or incorrect readings.

4. Availability and Performance Monitoring

Continuously monitor system performance, server health and connctivity. Early detection of issues allows quick fixes before users are affected.

5. Regular Testing and Maintenance

perform regular system testing and keep software updated to fix bugs and vulnerabilities. This helps maintain long term dependability.

6. Safety Consideration

The system must prevent false alarms and incorrect location reporting that may cause unnecessary panic. Therefore, abnormal GPS jumps should be filtered and verified before notifying parents.

Question 4: Identify Hardware failures, Software failures, and Operational failures for your system (at least 2 for all)?

Answer 4:

In the CLMS system, different types of failures may occure during operation. We can sort these failure into Hardware failures, Software failures, and Operational failures. Identifying these risks is important to improve system dependability and ensure continuous service for parents and children.

1. **Hardware failures**
   Hardware failures refer to problems caused by physical devices or infrastructure components. Examples include:
   - **Smartphone device failure:** The child's smartphone may experience battery loss, GPS sensor malfunction, or physical damage. As a result, the system may not be able to collect or transmit location data.
   - **Server or storage failure:** The backend server or database hardware may fail due to disk crashes or power interruptions. This can lead to service downtime or loss of stored location history.

2. **Software failures**
   Software failures occur when system applications or programs do not work correctly. Examples include:
   - **Mobile application errors:** The tracking application on the child's phone may crash, stop running in the background, or lose location permissions. This prevents regular GPS updates from being sent.
   - **Backend processing bugs:** Errors in the server software, such as incorrect geofencing logic or API failures, may cause wrong notifications or block parents from accessing location information.

3. **Operational failures**
   Operational failures are caused by human mistakes or improper system management. Examples include:
   - **Configuration and deployment mistakes:** Incorrect server settings, firewall rules, or failed updates may stop the system from functioning properly.
   - **User or administrator errors:** Parents may be linked to the wrong child account, or access control may be set incorrectly, which can cause privacy issues or prevent authorized access.

Question 5: What are Redundancy and Diversity? According to your architecture, Identify redundancy and diversity issues need to be considered in your system (at least 2 issues)?

Answer 5:

Redundancy means adding backup components or duplicated resources so that the system can continue operating even if one component fails. It helps eliminate single points of failure.

Diversity means using different technologies, methods, or implementations to reduce the risk that the same type of failure affects multiple components at the same time (common-cause failure).

Based on the layered architecture shown in Question 2, the following redundancy and diversity issues should be considered

1. **Backend and Database Redundancy**

   In the current architecture, the backend services and database are critical components. If only one service node or one database instance is deployed, it becomes a single point of failure.

   To address this, multiple service nodes should be deployed behind a load balancer. If one node crashes due to hardware or software failure, other nodes can continue processing requests.

   Similarly, the database should use replication (e.g., master–slave configuration). If the master node fails, a replica can take over, ensuring data availability and preventing service interruption.

2. **Location and Platform Diversity**

   The CLMS system heavily relies on GPS data and push notifications. However, GPS signals may be weak or unavailable indoors, and push notifications may fail due to network issues.

   To improve diversity:

   - The system can combine multiple location sources such as GPS, Wi-Fi positioning, and cellular data to improve accuracy and reduce dependency on a single method.

- For SOS alerts and notifications, the system can support multiple communication channels (e.g., push notifications and SMS). If one channel fails, the other can still deliver important alerts to parents.

By applying both redundancy and diversity in the architecture, the CLMS system reduces single points of failure and minimizes the risk of common-cause failures, thereby improving overall dependability.

Question 6: Specify availability and reliability requirements for your system (at least 2 requirements)?

Answer 6:

Functional Requirement (Reliability — Data Integrity): The child device (mobile app or embedded GPS tracker) must buffer GPS location samples locally whenever the network connection is lost and forward them to the backend as soon as connectivity is restored. The backend must accept location samples whose recorded timestamp is older than the receive time and store them in chronological order, so that gaps in connectivity do not produce gaps in the parent-visible location history. This is supported in the current implementation by the GPS validation pipeline, which normalises both seconds- and millisecond-precision epoch timestamps before insert.

Non-functional Requirement (Availability): The CLMS backend system must achieve a minimum uptime of 99.9% during active monitoring hours (06:00 to 22:00). This ensures that parents can reliably access the "View child's current location" feature and the zone enter/exit notification feed during the times children are away from home.

Question 7: Redesign your system to take Fault-tolerant architectures into consideration?

Answer 7:

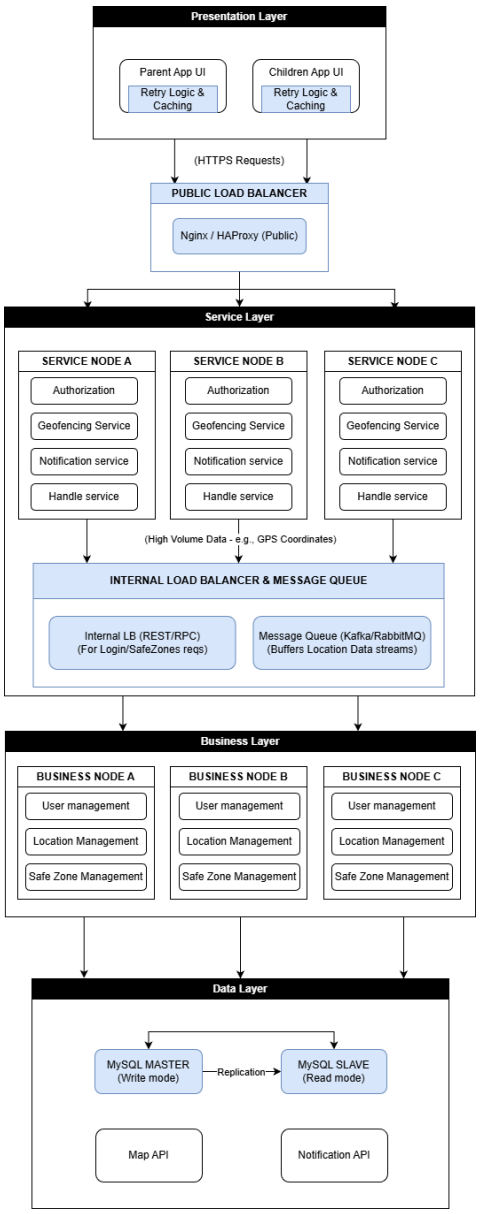


Figure: Fault-tolerant architecture

To address the hardware and software failures identified in Question 4, the architecture has been upgraded with the following fault-tolerant mechanisms:

- Load Balancing Layer: A Public Load Balancer (Nginx/HAProxy) is introduced to distribute incoming traffic across multiple Service Nodes. This ensures that if one server fails (Hardware Failure), the traffic is automatically redirected to a healthy node, preventing service downtime.
- Redundant Service and Business Clusters: The Service and Business layers are no longer single points of failure. They are deployed as clusters (Node A, Node B, Node C). If a specific node crashes, others in the cluster continue to process requests.

- Message Queue (Decoupling): An internal Message Queue (Kafka/RabbitMQ) is placed between the Service and Business layers. This acts as a buffer for high-frequency GPS data. If the database or Business layer is temporarily overloaded (Software Failure), the queue holds the data until it can be processed, preventing data loss.
- Database Replication: The Data Layer now uses a Master-Slave configuration. The Master node handles write operations, while data is replicated to Slave nodes. In the event of a disk failure on the Master, a Slave node can take over, ensuring data durability and continuous availability.

SECTION 2. Safety Engineering

Question 8: What are hazard and accident? Identify 2 hazards, 2 accidents, 2 damages with your system?

<u>Answer 8:</u>

Hazard is a condition with the potential for causing or contributing to an accident. A failure of the sensor that measures blood glucose is an example of a hazard. ***Accident*** is an unplanned event or sequence of events which results in human death or injury, damage to property, or to the environment. An overdose of insulin is an example of an accident.

Hazards in the CLMS System:

1. Incorrect location detection: The system may report an incorrect GPS location due to weak signal, sensor errors, or software bugs.
2. Failure to send emergency (SOS) alerts: The system may fail to transmit an SOS alert due to network failure, application crash, or server malfunction.

Accidents in the CLMS System:

1. Parents cannot locate a child in a dangerous situation: If the system reports an incorrect location or fails to update location data, parents may not be able to find the child during an emergency.
2. Emergency alert not received in time: If the SOS alert fails to reach the parent, the child may remain in danger without timely assistance.

Damages in the CLMS System:

1. Risk to child safety: The child may experience harm or remain in a dangerous situation because parents cannot respond quickly.
2. Loss of user trust and reputation damage: Parents may lose trust in the CLMS system if it fails during critical situations.

Question 9: Build a table as in slide 21/Chap 3 for the hazards in question 7

<u>Answer 9:</u>

[Answer here]

Identified Hazard	**Hazard Probability**	**Accident Severity**	**Estimated Risk**	**Acceptability**

1. Incorrect location detection: The system reports an incorrect GPS location due to weak signal, sensor errors, or software bugs	Medium	High	High	Intolerable
2. Failure to send SOS alert: The system fails to transmit an SOS alert due to network failure, application crash, or server malfunction	Low	High	Medium	ALARP

Hazard 1: Incorrect Location Detection

Probability: Medium. GPS signal dropout is a realistic daily occurrence, especially inside buildings or along dense urban streets like those commonly found in Ho Chi Minh City. Software bugs in the location computation layer also contribute to this likelihood.

Severity: High. A falsely reported location during an emergency means parents will search in the wrong place entirely. The time window to reach and help a child in danger closes very quickly when responders are heading the wrong direction.

Estimated Risk: High. A medium probability combined with high real-world consequences pushes the overall risk into the high band. Neither factor is low enough to offset the other.

Acceptability: Intolerable. This hazard cannot be left without mitigation. The system must implement multi-source positioning such as combining GPS with Wi-Fi and cellular data, along with input validation to filter out abnormal location jumps before displaying or acting on them.

Hazard 2: Failure to Send SOS Alert

Probability: Low. The SOS feature is simple and manually triggered. A failure requires both a network outage and an application crash to happen at the same time, which is an unlikely combination under normal operating conditions.

Severity: High. A missed SOS in a genuine emergency leaves a child in danger with no adult aware of the situation. Depending on the circumstances, the consequences could be life-threatening.

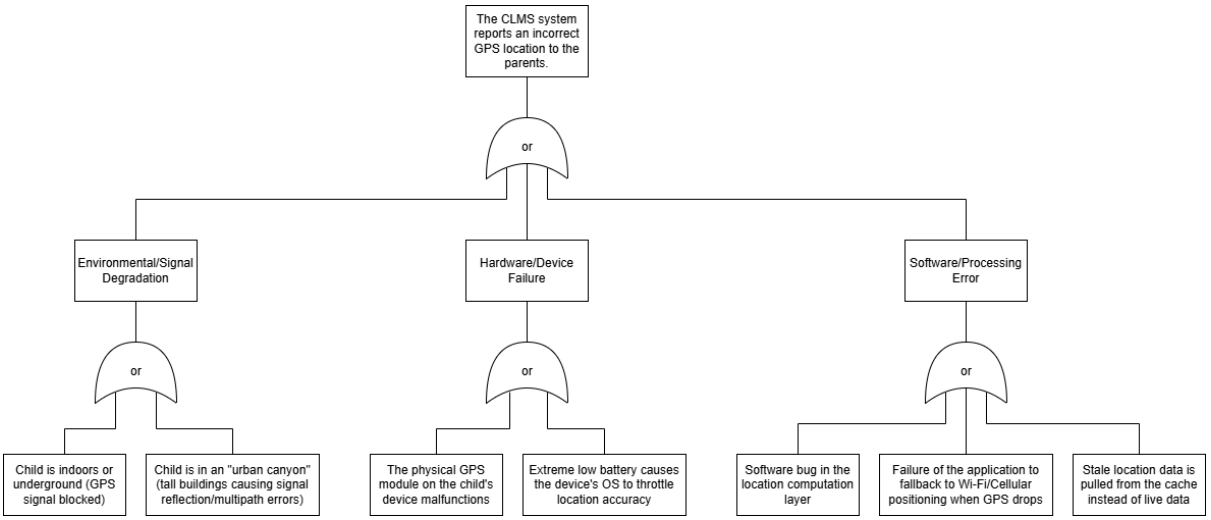
Estimated Risk: Medium. The low probability keeps this hazard out of the intolerable band, but the potential severity is serious enough that it still demands active attention and concrete mitigation measures.

Acceptability: ALARP. The risk must be reduced as far as reasonably practicable. Practical mitigations include adding an SMS fallback channel for alerts and implementing local queuing so the SOS is sent automatically once connectivity is restored.

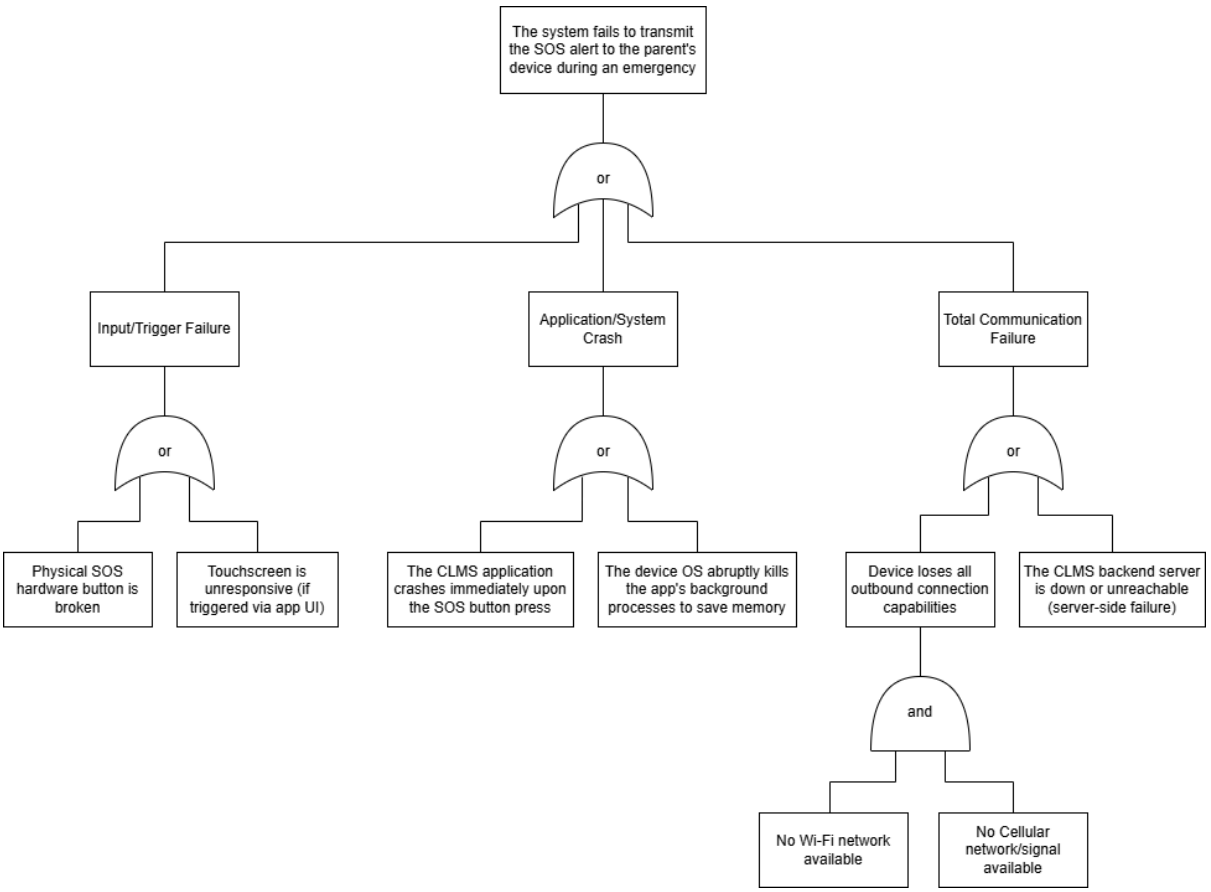
Question 10: Choose 2 hazards in question 8 and draw the fault trees for it

<u>Answer 10:</u>

Fault Tree 1: Incorrect Location Detection



Fault Tree 2: Failure to Send SOS Alert



Question 11: After analysing the hazards as in question 9, let write 2 safety requirements for your system

<u>Answer 11:</u>

[Answer here]

Safety requirements 1: Prevent incorrect location reporting

The CLMS system should validate and cross-checking the location data before displaying it to parents or using it for geomentric decisions. If GPS data is inconsitent, outdated or abnormal, the system should reject it or mark it unrealuable instead of treating it as the usual.

This requirement is necessary because incorrect location information may cause parents to search in the wrong place during emergency. To support this requirement, the system should apply timestamp checking, abnormal jump filtering, and combine multiple location sources such as GPS, WiFi, cellular postioning to improve accuracy.

Safety requirements 2: Failure to send SOS alert

The CLMS system shall ensure reliable delivery of SOS alerts by using retry and fallback mechanisms. If the primary communication channel fails, the system shall automatically resend the alert through an alternative channel, such as SMS, and continue retrying until the alert is successfully delivered or stored for immediate follow-up.

This requirement is important because a missed SOS alert may leave the child in danger without timely assistance from parents. To satisfy this requirement, the system should support local queuing, repeated transmission attempts, and multiple communication channels for emergency notifications.

In conclusion, these safety requirements help reduce the likelihood and impact of critical hazards in the CLMS system. By validating location information and improving the dependability of SOS alert delivery, the system can provide safer and more trustworthy services for both children and parents.

Question 12: Choose 2 safety requirements in question 6 (that we now call 'claim'), let identify 3 evidences that support each claim.

<u>Answer 12:</u>

Claim 1: Functional Requirement (Reliability - Data Integrity)

Evidence 1: Automated unit and integration test reports verifying that the mobile application successfully writes GPS coordinates to local storage (SQLite) when the network interface is manually disabled.

Evidence 2: A formal code audit and design document confirming the implementation of an event listener that triggers the synchronization of cached data as soon as internet connectivity is restored.

Evidence 3: Comparative data logs from a 24-hour field test showing a 100% match between the count of GPS pings recorded on the local device and the pings finally stored on the server after periods of signal loss.

Claim 2: Non-functional Requirement (Availability)

Evidence 1: Historical uptime reports generated by system monitoring tools (e.g., Grafana or Prometheus) showing that the backend achieved at least 99.9% availability during the 06:00 to 22:00 window over a 90-day period.

Evidence 2: Documentation of successful "Load Balancer Failover" tests where traffic was automatically and seamlessly redirected to healthy service nodes when a primary node was intentionally shut down.

Evidence 3: Database replication audit logs from the data layer verifying that the Master-Slave configuration maintained real-time synchronization, allowing for immediate data durability and service continuity.

SECTION 3. Security Engineering

Question 13: In your opinion, what are *Confidentiality, Integrity,* and *Availability* of a system? Let give 3 examples in which your system violates the Confidentiality, Integrity, and Availability, respectively

<u>Answer 13:</u>

1. Confidentiality: Ensuring that sensitive information is accessed only by authorized parties.
2. Integrity: Ensuring that data is accurate, complete, and has not been modified by unauthorized users or through accidental corruption.
3. Availability: Ensuring that systems, networks, and data are consistently accessible to authorized users when needed.

Examples of Violations:

1. Confidentiality Violation: An attacker can change the childId parameter in a request such as GET /api/dashboard/:childId or GET /api/location/latest/:childId to read the real-time location of any child registered in the system, because these read endpoints do not currently enforce parent-to-child ownership. The same parameter substitution would expose location history and per-child safe zones.
2. Integrity Violation: An attacker positioned on the network path between the child device (or the cloud IoT broker that forwards data via webhook) and the CLMS backend performs a Man-in-the-Middle attack, intercepts a GPS payload in transit, and rewrites the coordinates so that a child appears to be at "School" while actually elsewhere. This is possible whenever the transport is plain HTTP rather than HTTPS.
3. Availability Violation: A malicious actor launches a Distributed Denial of Service (DDoS) attack against the CLMS backend, causing the location ingest and dashboard APIs to time out. Parents lose visibility of their child's current location and stop receiving zone enter/exit notifications during the outage.

Question 14: List 3 assets that need to be protected in your system? List also 3 for each Vulnerability, Attack, Threat, Control for your system (Chap 4)?

<u>Answer 14:</u>

3 Assets to Protect in CLMS:

1. Child’s Real-time Location Data: The most sensitive asset; unauthorized access poses a physical safety risk.

2. Parental Account Credentials: Access to these allows an attacker to monitor or spoof any child linked to the account.
3. Historical Movement Logs: Stored data that reveals a child's daily routines (home, school, playground).

Category	Item 1	Item 2	Item 3
Vulnerability	Lack of brute-force protection on OTP verification (6-digit code with no rate limit on /auth/verify-otp)	Lack of HTTPS enforcement at the application layer (the backend serves over plain HTTP and relies on the deployment proxy for TLS)	Missing object-level authorisation on read endpoints (/api/dashboard/:childId, /api/location/latest/:childId, /api/safezones/:childId)
Attack	OTP brute-force: an attacker iterates the 6-digit code space within the 5-minute OTP validity window to gain unauthorised account access	SQL injection: attempting to inject malicious payloads through user-supplied parameters into database queries	Session hijacking: stealing a parent's JWT (e.g. via XSS or unprotected browser storage) to impersonate them in subsequent API calls
Threat	Stalking: an unauthorised person tracks a child's movements through a compromised parent account or the unprotected :childId API surface	Service Disruption: a malicious actor overloads the backend so location updates and zone-event notifications are delayed or lost during an emergency	Data Breach: exposure of personal family data, location history, and safe-zone definitions, leading to identity theft or targeted physical risk
Control	Multi-Factor Authentication (MFA): Requiring a code sent to the parent's phone for login.	TLS/SSL Encryption: terminate HTTPS at the deployment edge so all communication between the parent's browser, the cloud IoT broker, and the backend is encrypted	Input validation, parameterized SQL, and per-child object-level authorisation: validate every GPS payload (range, scale, timestamp), rely on parameterized queries throughout, and require

			a JWT plus parent-to-child ownership check on every read endpoint

Question 15: Analyse the assets and threats (question 13) based on slides 32, 33 of Chap 4?

<u>Answer 15:</u>

Part A: Asset Analysis

Asset	Value	Exposure
Real-time Location Data	High. Essential for the primary function of the system. Safety-critical.	High. If exposed, an unauthorised person can track the child, leading to physical danger and legal liability.
Parental Credentials (Google identity + OTP)	High. Required to manage accounts and access all child data.	High. Compromise leads to total account takeover, allowing stalking or spoofing of the child's location.
Zone-event and SOS notification channel	High. Critical for surfacing zone enter/exit events and emergency SOS alerts triggered by the child.	High. Failure to deliver alerts may leave the parent unaware of dangerous transitions or emergencies.
Historical Movement Logs	Medium-High. Reveals daily routines and patterns of a child's movements.	Medium. Exposure can enable predictive stalking or profiling of the child's regular locations.

Part B: Threat and Control Analysis

Threat	Probability	Control	Feasibility
Unauthorised user gains access to a child's location via	High (currently exploitable:	Add JWT-based requireAuth middleware and an object-level	High. The existing requireAuth middleware (already used on

the open :childId API surface	/api/dashboard/:childId is unauthenticated)	check that the JWT subject is the parent linked to :childId	/api/settings) just needs to be extended to read endpoints.
System becomes unavailable due to DDoS attack	Low	Public load balancer with DDoS protection, server clustering, message queue between ingest and DB (as in the redesigned Q7 architecture)	Medium. Technically feasible; adds monthly infrastructure cost.
Attacker intercepts data in transit (MitM) and alters GPS coordinates	Medium	Enforce TLS 1.3 for traffic between the parent's browser, the cloud IoT broker webhooks, and the backend	High. Standard practice; terminated at the deployment edge.
OTP brute-force / replay against the email-based MFA step	High	Rate-limit /auth/verify-otp (e.g. 5 attempts per OTP validity window) and invalidate the OTP after the threshold	High. Small change with express-rate-limit; minimal friction for legitimate users.

Question 16: Specify 3 security requirements for your system (based on the analysis in question 14)

<u>Answer 16:</u>

1. Data Encryption in Transit (Confidentiality): All communication between the child device (mobile app or embedded GPS tracker), the cloud IoT broker, the parent's web interface, and the backend server must be encrypted using TLS 1.3 or higher. This ensures that GPS coordinates and notifications cannot be read or intercepted by attackers on unsecured networks such as public Wi-Fi.
2. Strong Authentication and Access Control (Authentication): The system shall use Google OAuth combined with a 6-digit one-time password emailed to the parent as Multi-Factor Authentication for all parent accounts. To prevent OTP brute-force, the /auth/verify-otp endpoint must be rate-limited (e.g. 5 attempts

per 5-minute OTP validity window); once the threshold is exceeded, the issued OTP is invalidated and a new one must be requested. All authenticated read endpoints must additionally enforce object-level authorisation, ensuring a parent may only access children linked to their own account.

3. API Security and Input Validation (Integrity): The backend server must validate and normalise all incoming GPS payloads (latitude/longitude range, timestamp scale, scaled-integer denormalisation) before persistence. Every webhook or polled request containing GPS data must authenticate the sender — for the cloud IoT broker this is achieved via OAuth2 client_credentials with per-tenant client ID/secret pairs (already implemented); for any future child mobile-app variant, the request must carry a per-device API token. All database access must use parameterized queries so user-supplied parameters cannot alter query semantics.